

JAVA INTERVIEW PREPARATION

CHAPTER 22

JAVA MEMORY MODEL, SYNCHRONIZED, AND VOLATILE

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

The Java Memory Model, synchronized, and volatile: Reasoning About Shared State

Senior / Lead Java Developer Reading Chapter

Concurrency bugs are difficult because source-code order is not the whole execution model. Compilers optimize, processors reorder and cache memory operations, and threads interleave at many legal points. The Java Memory Model, usually called the JMM, defines which values a thread is allowed to observe and which synchronization actions create reliable ordering between threads.

The JMM does not describe a particular CPU cache. It is a language-level contract that lets the JVM optimize while giving correctly synchronized programs portable behavior.

Three Separate Problems

Shared-memory correctness requires reasoning about atomicity, visibility, and ordering.

atomicity	- is an operation observed as one indivisible action?
visibility	- when does one thread observe another thread's write?
ordering	- which actions are guaranteed to occur before other actions?

These are related but not interchangeable. A write can be atomic yet not promptly visible. A value can be visible while a compound check-and-update remains racy.

```
if (stock > 0) {  
    stock--; // read, decision, and write are not one atomic transaction  
}
```

Two threads can both read one item in stock and both decrement it. Making the field visible does not protect the invariant.

Data Races and Happens-Before

A data race exists when different threads access the same variable, at least one access is a write, and the accesses are not ordered by a happens-before relationship. The result may be legal under the JMM but surprising to a developer reasoning only from wall-clock time.

Happens-before is a visibility and ordering guarantee. If action A happens-before action B, the effects of A are visible to B, and A is ordered before B for JMM reasoning.

Important relationships include:

- Program order within one thread.
- Unlocking a monitor happens-before a later successful lock of that monitor.
- A volatile write happens-before a later volatile read of the same variable.
- Actions before Thread.start happen-before actions in the started thread.
- All actions in a thread happen-before another thread successfully returns from join.
- Completing a Future establishes the documented visibility relationship with a successful get.
- Relationships are transitive.

Thread A	Thread B
write configuration	
unlock monitor	-----HB--> lock same monitor
	read configuration safely

Happens-before is not necessarily chronological tracing language. It is the formal ordering that makes communication reliable.

What synchronized Guarantees

Every Java object can act as a monitor. Entering a synchronized block acquires that monitor; leaving releases it, including when an exception exits the block.

```
final class Inventory {
    private int available;

    synchronized boolean reserveOne() {
        if (available == 0) return false;
        available--;
        return true;
    }
}
```

The monitor provides mutual exclusion: only one thread executes code guarded by the same monitor at a time. It also provides visibility: writes made before monitor release are visible after a later acquisition of that monitor.

The phrase "same monitor" is essential. These methods do not coordinate:

```
synchronized (lockA) { state++; }
synchronized (lockB) { System.out.println(state); }
```

Both sides of an invariant must follow one locking protocol.

Instance and Static Synchronization

An instance synchronized method locks `this`. A static synchronized method locks the Class object. They are different monitors.

```
synchronized void updateInstance() { } // locks this
static synchronized void updateGlobal() { } // locks Example.class
```

Two different instances can execute their instance synchronized methods simultaneously. If state is static but guarded only by instance locks, the protection is incomplete.

Prefer a private final lock when exposing the object monitor would allow unrelated external code to participate in the locking protocol.

```
private final Object stateLock = new Object();
```

Keep Critical Sections Cohesive and Small

A critical section should cover the complete invariant but avoid unrelated slow work. Protecting only the final write is often too narrow; holding the lock across database or network I/O is usually too broad.

```
Reservation reserve(String id) {
    Reservation reservation;
    synchronized (stateLock) {
        reservation = allocateLocally(id);
    }
    auditClient.send(reservation); // remote call outside the lock
    return reservation;
}
```

Moving I/O outside a lock changes failure semantics, so the design may require an outbox, compensation, or state transition. Lock optimization must preserve the business transaction, not merely reduce waiting.

Java monitors are reentrant: a thread holding a monitor may enter another synchronized method guarded by the same monitor. Reentrancy prevents self-deadlock in nested calls, but deeply implicit locking can still be hard to maintain.

wait, notify, and the Condition Predicate

`wait` releases the monitor and suspends the thread. The thread must reacquire the monitor before returning. `notify` wakes one waiter; `notifyAll` wakes all current waiters. All must be called while owning that monitor.

Always wait in a loop because wakeups can be spurious and another thread may consume the condition first.

```
synchronized Message take() throws InterruptedException {
    while (queue.isEmpty()) {
        wait();
    }
    return queue.removeFirst();
}
```

Prefer higher-level utilities such as `BlockingQueue`, `CountDownLatch`, or `Condition` for most application code. Understanding monitor waiting remains important for interviews and legacy diagnostics.

volatile: Visibility Without Mutual Exclusion

A volatile write publishes the written value and earlier actions to a thread that later reads that same volatile variable. Volatile also restricts reorderings around the access.

```
final class ServiceLoop implements Runnable {
    private volatile boolean running = true;

    public void stop() {
        running = false;
    }

    public void run() {
        while (running) {
            performOneUnit();
        }
    }
}
```

Without a visibility mechanism, the worker is not guaranteed to observe the update as intended. Volatile is appropriate here because correctness depends on the latest independent flag value, not a compound relationship among several fields.

Why volatile Does Not Make Increment Atomic

Increment is a read-modify-write sequence.

Thread A reads 5	Thread B reads 5
Thread A writes 6	Thread B writes 6
expected 7, observed 6	

```
private volatile int count;
void increment() {
    count++; // still racy
}
```

Use a synchronized invariant, `AtomicInteger`, `LongAdder` for suitable statistics, or another ownership model. Volatile protects communication about a value; it does not combine several actions into one atomic update.

Safe Publication

An object should not become reachable by other threads before construction is complete. Safe publication can occur through a static initializer, volatile reference, lock, concurrent collection, thread-start relationship, or properly constructed immutable object with final fields.

```
final class Configuration {
    private final URI endpoint;
    private final int retries;

    Configuration(URI endpoint, int retries) {
        this.endpoint = endpoint;
        this.retries = retries;
    }
}
```

Final fields receive special initialization guarantees when `this` does not escape during construction. Final does not make referenced mutable objects immutable. A final List reference can still refer to a list that another thread mutates.

Avoid registering listeners, starting threads, or placing `this` into shared storage from a constructor. Those actions can expose a partially initialized object.

Double-Checked Locking

Lazy singleton initialization using double-checked locking requires a volatile reference.

```
private static volatile Client instance;

static Client instance() {
    Client result = instance;
    if (result == null) {
        synchronized (ClientHolder.class) {
            result = instance;
            if (result == null) {
                result = new Client();
                instance = result;
            }
        }
    }
    return result;
}
```

Volatile prevents publication from being observed independently of the initialization actions. In ordinary code, an enum singleton, eager initialization, dependency-injection container, or initialization-on-demand holder is often simpler.

Immutable Snapshots and Ownership

The easiest shared mutable state is the state that is not shared or not mutable. A common design builds a new immutable snapshot and swaps one volatile reference.

```
private volatile RoutingTable routing = RoutingTable.empty();

void refresh() {
    RoutingTable next = loader.loadImmutableTable();
    routing = next;
}

Route route(Request request) {
    return routing.match(request);
}
```

Readers need no lock because each snapshot is complete and immutable. The volatile write publishes it. This works when updates can replace the whole logical state and temporary old snapshots are acceptable.

Thread confinement is another strong option: one thread or actor owns mutable state, and other threads communicate by messages. This converts memory races into message-ordering and capacity questions.

Source Order Is Not a Cross-Thread Guarantee

Within one thread, Java must preserve the behavior that thread is allowed to observe. Across threads without synchronization, compilers and processors may reorder independent operations or keep values in registers and caches in ways that expose surprising results.

```
// Initially: data = 0, ready = false

// Thread A
data = 42;
ready = true;

// Thread B
if (ready) {
    use(data); // without synchronization, source order alone is insufficient
}
```

Making only `ready` volatile creates the required publication edge:

Thread A	Thread B
data = 42	
volatile write ready = true	--HB-----> volatile read ready == true
	read data == 42

Program order places the data write before the volatile write in Thread A. The volatile synchronization edge connects threads, and happens-before transitivity carries visibility to the later data read. `data` does not also need to be volatile when every access follows this publication protocol.

This idiom publishes actions that occurred before the volatile write. It does not automatically order mutations performed after publication.

Correct Synchronization Restores Simple Reasoning

The JMM provides a strong practical rule: correctly synchronized, data-race-free programs appear sequentially consistent. Developers can reason as though thread actions were interleaved in an order that respects each thread's program order, rather than manually simulating every compiler and CPU optimization.

This does not make business logic correct. Two operations can be individually synchronized and still violate a higher-level invariant if the lock is released between them. “Data-race-free” is a memory-safety foundation, not proof that reservation, accounting, or workflow rules are correct.

```
memory-model question: are conflicting accesses ordered?
business question:      is the complete invariant one atomic transition?
```

Monitor Ownership, Reentrancy, and Fairness

Entering a synchronized region requires ownership of its exact monitor. A thread unable to acquire it becomes **BLOCKED**. The JVM does not promise fair first-in-first-out monitor acquisition, so a recently arriving thread may acquire before a longer waiter.

Monitor exit occurs on normal return and exceptional exit. This prevents a thrown exception from permanently retaining the monitor, but application state can still be partially changed before the exception. Maintain invariants with careful update ordering or rollback logic.

```
synchronized (stateLock) {
    debit(source);
    credit(target); // if this throws, monitor is released but state may be partial
}
```

Reentrancy counts nested acquisition by the same thread. It avoids self-deadlock when one synchronized method calls another using the same monitor, but it can hide long critical sections across several method layers. Profiles should attribute contention to the full call path, not only the outermost `synchronized` keyword.

Lock Objects Are Part of the Encapsulation Boundary

Synchronizing on publicly reachable objects lets unrelated code join the locking protocol accidentally.

```
// Fragile choices
synchronized (this) { ... }           // callers can also lock this
synchronized (customerId.intern()) { ... } // global shared pool and retention risk
```

A private final lock keeps monitor identity stable and encapsulated. Never replace the object used as a lock while threads are active; different threads could then synchronize on different monitor instances while believing they protect the same state.

```
private final Object balanceLock = new Object();
```

Document which fields the lock guards. This turns synchronization from scattered syntax into a reviewable protocol: every read and write of the guarded invariant must participate.

Wait Sets: Notification Is Not State

`notify` does not store a permit for a future waiter. The durable truth is the condition predicate guarded by the monitor. A thread changes state while holding the monitor, then notifies; a waiter loops and rechecks state after reacquiring.

```
producer owns monitor
add item
condition queueNotEmpty becomes true
notifyAll
unlock
waiter wakes, reacquires, rechecks queueNotEmpty
```

Use `notifyAll` when different predicates share one monitor or when waking the wrong single waiter could leave progress stalled. It can create extra wakeups, but correctness comes first. Higher-level constructs separate conditions and encode interruption, timeout, and capacity behavior more clearly.

A timed wait must recompute remaining time in a loop. One `wait(timeout)` followed by assuming success confuses timeout, notification, spurious wakeup, and stolen conditions.

Volatile Supports State Publication, Not Coupled Invariants

Volatile works well for a status flag, generation number, or immutable snapshot reference. It is weak when several fields must change together.

```
volatile int lower;
volatile int upper;

// A reader can observe lower from the new update and upper from the old update.
```

Replacing both with one immutable object produces a coherent version:

```
record Range(int lower, int upper) {
    Range {
        if (lower > upper) throw new IllegalArgumentException();
    }
}

private volatile Range range = new Range(0, 100);
```

Readers see either the old Range reference or the new complete Range reference. This is often simpler than coordinating several volatile fields.

Safe Publication Patterns as a Memory Image

Construction and publication are separate phases:

construct privately	publish through HB edge	consume
allocate object		
initialize fields		
establish invariants	--> volatile write / lock /	read after matching edge
no this escape	static init / concurrent map	observe complete state

Common safe mechanisms include:

- Class initialization for static state.
- Writing the reference to a volatile field.
- Storing and reading under the same lock protocol.
- Publishing through a concurrency utility with documented memory effects.
- Calling `Thread.start` after preparing state consumed by the new thread.
- Final-field semantics for a properly constructed object that does not escape.

Plain assignment to a shared non-volatile field without another happens-before edge is not a safe general publication mechanism.

Final Fields and Constructor Escape

Final fields receive special JMM treatment when construction completes normally and the object does not escape during construction. A thread that obtains a properly constructed object can rely on the initialized final-field values under those rules.


```
final class Listener {
    private final URI endpoint;

    Listener(Registry registry, URI endpoint) {
        registry.register(this); // this escapes too early
        this.endpoint = endpoint;
    }
}
```

Another thread could call the registered object before `endpoint` is assigned. Avoid callbacks, registration, thread start, overridable method calls, and insertion into shared collections from constructors. Use a factory that constructs first and publishes second.

Final applies to the field reference, not the transitive object graph. A final array or list can still be mutated unless ownership prevents it.

Choosing synchronized, volatile, or Ownership

Choose from the invariant rather than relative performance folklore:

requirement	likely mechanism
independent stop/status flag	volatile
immutable configuration snapshot	volatile reference
read-modify-write counter	AtomicInteger or synchronized
several fields must change together	synchronized / Lock / ownership
producer-consumer handoff	BlockingQueue
one thread owns mutable aggregate	actor/event loop confinement
wait for one-time completion	latch, Future, or structured task

Modern JVMs optimize uncontended synchronization effectively; an incorrect volatile design is not faster in any meaningful engineering sense. Measure contention after selecting a correct protocol.

A Strong Interview Explanation

Explain the JMM as the language contract governing which writes a read may observe and how actions are ordered across threads. Separate atomicity, visibility, and ordering. Define a data race as conflicting accesses without happens-before ordering, then name the common edges: monitor unlock-to-lock, volatile write-to-read, thread start, and thread join.

State that `synchronized` provides mutual exclusion plus visibility when every access uses the same monitor. `volatile` provides publication and ordering for a variable but cannot make a compound update atomic. Discuss safe publication, final-field construction, and `this` escape. A senior answer ends with design preference: immutable snapshots, confinement, and higher-level concurrency utilities often produce simpler proofs than scattered low-level synchronization.

Production Perspective

Typical failures include volatile counters losing updates, different locks guarding the same invariant, unsafe publication of mutable configuration, blocking I/O inside a hot monitor, and swallowing InterruptedException in wait loops. CPU profiles may show monitor contention; thread dumps show BLOCKED threads and lock owners; Java Flight Recorder can reveal contention duration.

Do not add volatile randomly until a test passes. Document the invariant, identify every access, choose one communication mechanism, and verify that all participating code follows it. Race bugs that disappear under logging may be timing-sensitive rather than fixed.

Interview-Ready Summary

The JMM defines legal observations and synchronization ordering. Correctness requires separate reasoning about atomicity, visibility, and ordering. Happens-before relationships are established by monitors, volatile access, thread lifecycle actions, futures, and other specified synchronization points.

Synchronized provides mutual exclusion and visibility for code using the same monitor. Volatile provides visibility and ordering for a variable but does not make compound operations such as increment atomic. Safe publication prevents threads from observing partially initialized state. Prefer immutability, snapshots, and ownership when they reduce shared mutation.

Revision Notes

- The JMM is a language contract, not a diagram of one CPU cache.
- Atomicity, visibility, and ordering are separate concerns.
- A data race lacks a happens-before ordering between conflicting accesses.
- Unlocking a monitor happens-before a later lock of that monitor.
- A volatile write happens-before a later read of the same volatile variable.
- Thread.start and Thread.join create visibility relationships.
- synchronized combines mutual exclusion with visibility.
- Synchronization works only when accesses use the same monitor protocol.
- Instance synchronized methods lock this; static ones lock the Class object.
- Monitors are reentrant.
- Keep the complete invariant inside the critical section.
- Avoid slow remote I/O while holding locks when semantics allow.
- Always test a wait condition in a loop.
- Volatile is suitable for independent flags and published references.
- Volatile increment is still a non-atomic read-modify-write.
- Publish objects only after construction completes.
- Final-field guarantees require that this does not escape construction.
- Double-checked locking requires a volatile reference.
- Immutable snapshots plus a volatile reference can remove reader locking.
- Diagnose contention and ownership rather than adding synchronization blindly.
- Source-code order is not a cross-thread communication guarantee.
- A volatile publication flag can make earlier ordinary writes visible through transitivity.
- Correctly synchronized data-race-free programs appear sequentially consistent.
- Java monitors do not promise fair lock acquisition.
- Exceptions release a monitor but can still leave partially changed business state.
- Notification is not stored state; the guarded condition predicate is the durable truth.
- Publishing several coupled values is often safer through one immutable snapshot.
- Lock identity should be private, final, stable, and documented with guarded fields.
- Final-field guarantees do not make referenced mutable objects deeply immutable.